

CoffeeManager-OS

Project Roadmap & Implementation Plan

Hybrid Agent Architecture — Python + OpenClaw

Version 1.0 · May 2026

Prepared for: Coffee Shop Owner / Operations Lead

Executive Summary

CoffeeManager-OS is a hybrid autonomous operations agent purpose-built for a high-volume specialty coffee shop in Lithuania. It combines deterministic Python logic for all financial calculations with the OpenClaw agentic brain for non-deterministic tasks such as government portal navigation, OCR receipt processing, and staff compliance workflows.

This document defines the full implementation roadmap across five phases, from environment setup through to production deployment and ongoing optimisation. The total estimated build time is 14–18 weeks for a solo developer or 8–10 weeks with a two-person team.

Key Principle

Python handles all arithmetic and data formatting. OpenClaw handles all judgment, navigation, and document understanding. These two concerns never cross.

Project Goals

- Full VMI i.EKA / i.SAF compliance with zero manual data entry
- Sodra new-hire registration drafted within 60 seconds of detection
- Daily Z-report audit running autonomously at 07:00 every morning
- Inventory reorder triggered automatically when stock falls below safety threshold
- 100% human-in-the-loop on all tax submissions and employee registrations

Technology Stack

Layer	Technology	Purpose
Deterministic core	Python 3.11+	VAT logic, P&L, inventory, i.SAF XML
Agentic brain	OpenClaw	Portal navigation, OCR, HR forms
OCR engine	docTR / EasyOCR	Receipt and invoice scanning
Database	SQLite (WAL mode)	Audit log, portal actions, notifications
Web framework	FastAPI	Dashboard, webhooks, POS integration
Notifications	Telegram Bot API	Owner alerts, HITL confirmations
POS connector	Paysera / RoboLabs API	Sales data, Z-report extraction

Phase 1 Environment & Foundation

Weeks 1–2

Set up the full development environment, project structure, database schema, and configuration management before writing any business logic. A clean foundation here prevents rework in every subsequent phase.

Step-by-step tasks

1. Create the project directory structure (core/, agent/, data/, config/, skills/)
2. Initialise a Python 3.11+ virtual environment and pin all dependencies in requirements.txt
3. Install Node.js for any docx generation scripts; confirm npm package availability
4. Run migrate.py to initialise memory.sqlite with all four schema migrations
5. Create config/settings.env with placeholders for all required credentials
6. Register a Telegram Bot via @BotFather; obtain BOT_TOKEN and CHAT_ID
7. Obtain OpenClaw API key from the OpenClaw dashboard
8. Confirm POS API credentials (Paysera or RoboLabs) and test a sample Z-report pull
9. Write and run a smoke-test script that reads from SQLite, sends a Telegram message, and calls the POS API
10. Commit the skeleton to version control with a .gitignore excluding config/ and data/

Deliverables

Task	Owner	Effort	Dependency
Project scaffold created	Developer	2 hrs	—
migrate.py executed successfully	Developer	1 hr	Scaffold
Telegram bot live and tested	Developer	1 hr	Credentials
POS API smoke test passing	Developer	2 hrs	POS credentials
Environment smoke test green	Developer	1 hr	All above

Risk

POS API documentation for Lithuanian providers (Paysera/RoboLabs) can be sparse. Budget an extra day if test credentials are not immediately available.

Phase 2 Python Deterministic Core

Weeks 3–5

Build and test all deterministic Python modules. These must be correct before the OpenClaw agent is introduced, because the agent trusts Python output completely and never re-validates it.

Step-by-step tasks

11. Implement core/accounting.py: Decimal-based PVM calculation (21% standard, 9% reduced), Z-report audit logic, and run_daily_audit() returning a typed dict
12. Implement core/inventory.py: check_stock_levels() pulling from POS API and comparing against configurable safety-stock thresholds
13. Implement core/integrations/paysera.py (or robolabs.py): authenticated API client with retry logic and structured response models
14. Implement core/isaf_generator.py: build_isaf_xml() producing valid i.SAF XML validated against the official VMI XSD schema file
15. Download the official VMI i.SAF XSD from the VMI developer portal and add to config/schemas/isaf.xsd
16. Write unit tests for every function in core/ using pytest; target 100% coverage on accounting.py
17. Implement migrate helper functions: log_audit(), log_portal_action(), notify_if_new()
18. Add a CLI test runner: python -m core.accounting --test to verify Z-report logic against fixtures

Deliverables

Task	Owner	Effort	Dependency
accounting.py with full PVM logic	Developer	1 day	—
inventory.py with API integration	Developer	1 day	POS API
isaf_generator.py + XSD validation	Developer	2 days	VMi XSD
pytest suite, all tests green	Developer	1 day	All above
migrate helpers integrated	Developer	0.5 day	migrate.py

Critical Rule

Never use float arithmetic for any monetary value. All amounts must use Python's Decimal type with ROUND_HALF_UP. A single float rounding error in a tax filing can trigger a VMi audit.

Phase 3 OpenClaw Agent Brain

Weeks 6–9

Implement the OpenClaw agentic layer: the soul.md system prompt, all skill files, and the agent runner that bridges Python context to OpenClaw tasks. This phase ends with the full hybrid loop running end-to-end in a staging environment.

Step-by-step tasks

19. Write agent/soul.md: full identity, tool manifest, protocol definitions for Daily Audit, Receipt Processing, New Hire, and Inventory tasks
20. Implement agent/runner.py: OpenClawAgent class with run_task() dispatcher, session ID management, and structured result returns
21. Implement skills/scan_receipt.py: docTR OCR pipeline, field extraction (VAT code, company code, date, amounts, PVM code), confidence scoring
22. Implement skills/vmi_submit.md: step-by-step OpenClaw instructions for navigating imas.vmi.lt, populating the i.SAF draft form, and saving without submitting
23. Implement skills/sodra_draft.md: OpenClaw instructions for draudejai.sodra.lt, 1-SD form population, and 24-hour advance-notice date enforcement
24. Implement main.py: full bootstrap sequence, Phase 1 Python core, Phase 2 OpenClaw dispatch, send_telegram() with deduplication
25. Wire notify_if_new() into every alert path: AUDIT_MISMATCH, STOCK_LOW, RECEIPT_READY, NEW_HIRE
26. Test the full hybrid loop end-to-end with a dummy receipt image and a dummy shift CSV entry in a staging environment with test credentials
27. Validate that no tax form or Sodra submission fires without Telegram confirmation

Deliverables

Task	Owner	Effort	Dependency
soul.md finalised and reviewed	Developer + Owner	1 day	—
agent/runner.py with full dispatcher	Developer	2 days	soul.md
scan_receipt.py OCR pipeline	Developer	2 days	docTR install
vmi_submit.md skill tested on portal	Developer	2 days	VMI test account
sodra_draft.md skill tested on portal	Developer	1 day	Sodra test account
main.py full loop, end-to-end test	Developer	1 day	All above

Security Note

VMI and Sodra credentials must never appear in soul.md or any skill file. All credentials are read from environment variables at runtime. OpenClaw receives only session tokens, not raw passwords.

Phase 4 FastAPI Dashboard & Hardening

Weeks 10–12

Add a lightweight FastAPI web dashboard for the owner, implement the POS webhook receiver, harden error handling across all modules, and perform a full compliance review before production.

Step-by-step tasks

28. Set up FastAPI app in api/main.py with endpoints: GET /status (last audit result), GET /receipts (processing queue), POST /webhook/pos (POS event receiver), POST /confirm/{event_key} (owner HITL confirmation)
29. Implement the /confirm/{event_key} endpoint to mark notifications_sent.acknowledged_at and trigger the next agentic step
30. Add APScheduler (or similar) for the 07:00 daily audit cron and the Sunday inventory check
31. Implement comprehensive error handling: every external API call and every OpenClaw task wrapped in try/except with structured error logging to portal_actions
32. Add circuit-breaker logic for VMI portal interactions: if three consecutive portal_actions show outcome=failure for the same portal, pause and alert the owner instead of retrying endlessly
33. Conduct a full compliance review: verify i.SAF XML validates against current VMI XSD, verify all Sodra 1-SD fields match current form version, verify all Telegram messages include required human-confirmation language
34. Write integration tests covering: audit mismatch detection, receipt OCR-to-draft pipeline, new hire detection-to-Sodra-draft flow
35. Deploy to a staging server (VPS or Raspberry Pi on-site) and run for one full week against live POS data with real receipts

Deliverables

Task	Owner	Effort	Dependency
FastAPI dashboard with all endpoints	Developer	3 days	Phase 3 complete
Cron scheduler configured	Developer	0.5 day	FastAPI
Circuit breaker for portal failures	Developer	1 day	runner.py
Compliance review sign-off	Developer + Owner	1 day	All modules
Integration test suite green	Developer	1 day	All above
One-week staging run completed	Developer + Owner	1 week	Staging deploy

Go / No-Go Gate

Do not proceed to Phase 5 until the staging run completes with: zero unhandled exceptions in portal_actions, zero unsolicited VMI or Sodra submissions, and owner confirmation that all Telegram alerts are accurate.

Phase 5 Production Launch & Optimisation

Weeks 13–18

Deploy to production, train the owner on the HITL confirmation workflow, and implement the optional enhancement modules (predictive scheduling, waste analytics, machine self-diagnosis) as the system stabilises.

Step-by-step tasks

36. Switch all environment variables from staging to production credentials
37. Run `migrate.py` on the production database to initialise the live schema
38. Deploy the FastAPI app behind a reverse proxy (nginx) with HTTPS and basic auth on the dashboard
39. Run the first live daily audit at 07:00 and verify Telegram output with the owner present
40. Train the owner on the HITL workflow: what each Telegram message means, how to confirm via the `/confirm` endpoint or Telegram reply, what to do if a message does not arrive
41. Monitor `portal_actions` and `audit_log` daily for the first two weeks; review with the owner each Friday
42. After two stable weeks, begin Phase 5 enhancements in order of ROI: (1) predictive labor scheduling, (2) supply chain waste analytics per barista, (3) customer sentiment loop, (4) machine self-diagnosis
43. Schedule a monthly review: check VMI XSD for schema updates, check Sodra portal for 1-SD form version changes, update skill files accordingly

Enhancement modules — priority order

#	Module	Est. effort	Primary benefit
1	Predictive labor scheduling	1–2 weeks	Reduce labor cost by 5–10%
2	Waste analytics per barista	1 week	Identify over-pouring; cut COGS
3	Customer sentiment loop	1 week	Win-back lapsed customers
4	Machine self-diagnosis	2–3 weeks	Prevent rush-hour breakdowns

Risk Register

The following risks are ranked by likelihood. All mitigations are built into the implementation plan above.

Risk	Impact	Mitigation
VMI portal UI changes breaking skill files	High	Monthly skill file review; screenshot-based regression test saved per session
i.SAF XML schema update by VMI	High	XSD pinned in config/; automated schema diff check in CI pipeline
Sodra 1-SD form version change	High	Monthly compliance review step in Phase 5 checklist
POS API downtime during audit window	Medium	Retry with exponential backoff; alert owner if 3 retries fail
OCR confidence below threshold on messy receipts	Medium	Confidence score gate: receipts below 80% flagged for manual review
Telegram bot token expiry or rate limit	Low	Token refresh documented in runbook; fallback email alert configured

Timeline Summary

P h a s e	Name	Weeks (solo)	Weeks (2-person)	Key output
1	Environment & Foundation	1–2	1	Running smoke tests
2	Python Deterministic Core	3–5	2–3	pytest suite green
3	OpenClaw Agent Brain	6–9	4–6	End-to-end hybrid loop
4	Dashboard & Hardening	10–12	7–8	Staging run complete
5	Production & Optimisation	13–18	9–10	Live + enhancements

Final Note

The HITL constraint — never submit a VMI filing or Sodra registration autonomously — is non-negotiable at every phase. Every release must be tested specifically for the absence of autonomous submission before being promoted to production.